🧪 Lab3 — MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

นายกันตวิชญ์ นาคนวล 673380027-4 รันสคริปต์ Python และ Wireshark
Scenario 4,5
นางสาวรติมา สวัสดิ์นที 673380055-9 สร้าง Topology ตั้งค่าเครื่อง และ switch
Scenario 1,2
นางสาวกิติญาดา กองคำ 673380509-6 ตั้งค่า Router และ Router, Switch
Configuration Scenario 3

📘 Lab Overview

Scenario

A client in one VLAN transfers files to a server in another VLAN using a custom MIME-based protocol (JSON header + binary payload). You will verify connectivity, observe encapsulation, and analyze packets using Wireshark.

This lab reinforces how Layer 2 segmentation, Layer 3 routing, TCP reliability, and application metadata interact.

🎯 Objectives

After completing this lab, you will be able to:

- Configure Router-on-a-Stick inter-VLAN routing
- Configure VLANs and trunking on a switch
- Deploy and run a MIME socket server and client
- Capture traffic using Wireshark
- Identify ARP, VLAN tags, TCP handshake, and application data

- Decode JSON headers in packet payload
- Troubleshoot network and application failures
- Document protocol behavior across OSI layers

🧰 Required Resources

- 1 Router (e.g., Cisco Systems ISR/virtual router)
- 1 Switch (Layer 2)
- 2 PCs or VMs
- Ethernet cables or virtual links
- Python 3 installed
- MIME file transfer project
- Wireshark installed (from Wireshark Foundation)

🗺️ Topology

```
None
PC1 (VLAN 10) ----\
                  \ access
                   \
                   [S1]---- trunk ----[R1]
                   /
                  / access
PC2 Server (VLAN 20)
```

📋 Addressing Scheme

Device	Interface	IP	Gateway
R1	G0/0.10	192.168.10.1/24	—
R1	G0/0.20	192.168.20.1/24	—
PC1-Client	NIC	192.168.10.10	192.168.10.1
PC2-Server	NIC	192.168.20.20	192.168.20.1

🧠 Background Theory

Router-on-a-Stick

A single physical interface carries multiple VLANs via 802.1Q tagging. Subinterfaces perform routing between broadcast domains.

MIME Metadata

MIME labels content type (e.g., image/png). Your protocol wraps files with JSON describing:

- MIME type
- Size

Why This Matters

Real systems — APIs, storage services, message brokers — depend on this exact pattern: structured metadata + binary payload over reliable transport.

🔧 Part 1 — Configure Network Devices

Step 1 — Router Configuration

```
None
enable
configure terminal
hostname R1

interface g0/0
no shutdown

interface g0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0

end
write memory
```

R1

Physical Config CLI Attributes

```
R1#
R1#
R1#
R1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#interface g0/0/0
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up

R1(config-if)#interface g0/0.10
%Invalid interface type and number
R1(config)#encapsulation dot1Q 10
                ^
% Invalid input detected at '^' marker.

R1(config)#interface g0/0/0.10
R1(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0/0.10, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0.10, changed state to up

R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip address 192.168.10.1 255.255.255.0
R1(config-subif)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#write memory
Building configuration...
[OK]
R1#
```

Copy Paste

Top

```
R1#
R1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#interface GigabitEthernet0/0/0.20
R1(config-subif)#encapsulation dot1Q 20
R1(config-subif)#ip address 192.168.20.1 255.255.255.0
R1(config-subif)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console
write memory
Building configuration...
[OK]
R1#
```

```
interface GigabitEthernet0/0/0
 no ip address
 duplex auto
 speed auto
!
interface GigabitEthernet0/0/0.10
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
!
interface GigabitEthernet0/0/0.20
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
!
interface GigabitEthernet0/0/1
 no ip address
 duplex auto
 speed auto
 shutdown
!
interface Vlan1
 no ip address
 shutdown
!
ip classless
!
ip flow-export version 9
!
!
!
!
```

Step 2 — Switch Configuration

```
None
enable
configure terminal
hostname S1

vlan 10
name CLIENTS

vlan 20
name SERVERS

interface g0/1
switchport mode trunk
```

```
interface fa0/1
switchport mode access
switchport access vlan 10

interface fa0/2
switchport mode access
switchport access vlan 20

end
write memory
```

S1

Physical Config CLI Attributes

```
Press RETURN to get started!


%LINK-5-CHANGED: Interface FastEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state to up

%LINK-5-CHANGED: Interface FastEthernet0/2, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/2, changed state to up


Switch>enable
Switch#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname S1
S1(config)#vlan 10
S1(config-vlan)#name CLIENTS
S1(config-vlan)#vlan 20
S1(config-vlan)#name SERVERS
S1(config-vlan)#interface g0/1
S1(config-if)#end
S1#
%SYS-5-CONFIG_I: Configured from console by console

S1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#interface g0/1
S1(config-if)#switchport mode trunk
S1(config-if)#interface fa0/1
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#interface fa0/2
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 20
S1(config-if)#end
S1#
%SYS-5-CONFIG_I: Configured from console by console

S1#write memory
Building configuration...
[OK]
S1#
```

Copy Paste

```
S1>enable
S1#show running-config
Building configuration...

Current configuration : 1201 bytes
!
version 15.0
no service timestamps log datetime msec
no service timestamps debug datetime msec
no service password-encryption
!
hostname S1
!
!
!
!
!
!
spanning-tree mode pvst
spanning-tree extend system-id
!
interface FastEthernet0/1
 switchport access vlan 10
 switchport mode access
!
interface FastEthernet0/2
 switchport access vlan 20
 switchport mode access
```

S1

Physical Config CLI Attributes

```
interface FastEthernet0/20
!
interface FastEthernet0/21
!
interface FastEthernet0/22
!
interface FastEthernet0/23
!
interface FastEthernet0/24
!
interface GigabitEthernet0/1
 switchport mode trunk
!
interface GigabitEthernet0/2
!
interface Vlan1
 no ip address
 shutdown
```

```
S1>enable
S1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
FastEthernet0/1        unassigned      YES manual up                    up
FastEthernet0/2        unassigned      YES manual up                    up
FastEthernet0/3        unassigned      YES manual down                  down
FastEthernet0/4        unassigned      YES manual down                  down
FastEthernet0/5        unassigned      YES manual down                  down
FastEthernet0/6        unassigned      YES manual down                  down
FastEthernet0/7        unassigned      YES manual down                  down
FastEthernet0/8        unassigned      YES manual down                  down
FastEthernet0/9        unassigned      YES manual down                  down
FastEthernet0/10       unassigned      YES manual down                  down
FastEthernet0/11       unassigned      YES manual down                  down
FastEthernet0/12       unassigned      YES manual down                  down
FastEthernet0/13       unassigned      YES manual down                  down
FastEthernet0/14       unassigned      YES manual down                  down
FastEthernet0/15       unassigned      YES manual down                  down
FastEthernet0/16       unassigned      YES manual down                  down
FastEthernet0/17       unassigned      YES manual down                  down
FastEthernet0/18       unassigned      YES manual down                  down
FastEthernet0/19       unassigned      YES manual down                  down
FastEthernet0/20       unassigned      YES manual down                  down
FastEthernet0/21       unassigned      YES manual down                  down
FastEthernet0/22       unassigned      YES manual down                  down
FastEthernet0/23       unassigned      YES manual down                  down
FastEthernet0/24       unassigned      YES manual down                  down
GigabitEthernet0/1     unassigned      YES manual up                    up
GigabitEthernet0/2     unassigned      YES manual down                  down
Vlan1                  unassigned      YES manual administratively down down
```

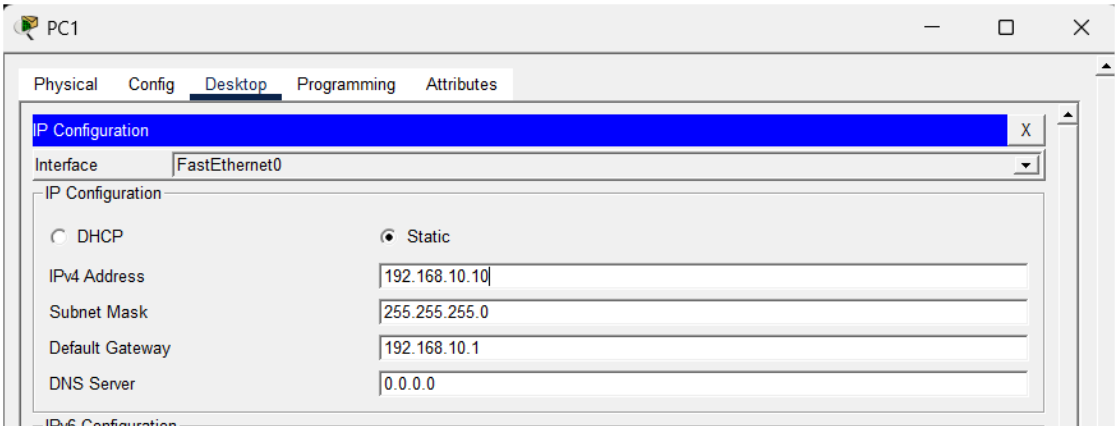
```
S1#show vlan brief

VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
1    default                          active    Fa0/3, Fa0/4, Fa0/5, Fa0/6
                                                Fa0/7, Fa0/8, Fa0/9, Fa0/10
                                                Fa0/11, Fa0/12, Fa0/13, Fa0/14
                                                Fa0/15, Fa0/16, Fa0/17, Fa0/18
                                                Fa0/19, Fa0/20, Fa0/21, Fa0/22
                                                Fa0/23, Fa0/24, Gig0/2
10   CLIENTS                          active    Fa0/1
20   SERVERS                          active    Fa0/2
1002 fddi-default                     active
1003 token-ring-default               active
1004 fddinet-default                  active
1005 trnet-default                    active
```

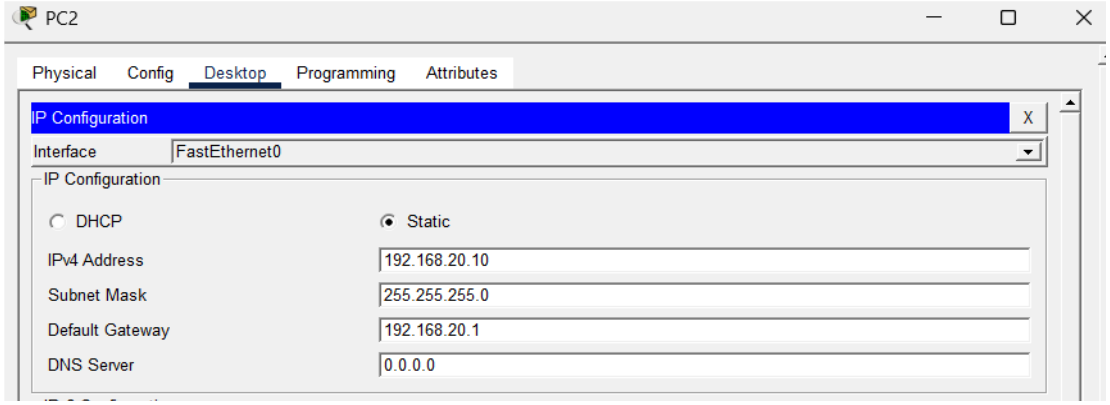
```
S1#show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Gig0/1      on           802.1q         trunking      1

Port        Vlans allowed on trunk
Gig0/1      1-1005

Port        Vlans allowed and active in management domain
Gig0/1      1,10,20

Port        Vlans in spanning tree forwarding state and not pruned
Gig0/1      1,10,20
```

```
S1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
FastEthernet0/1        unassigned      YES manual up                    up
FastEthernet0/2        unassigned      YES manual up                    up
FastEthernet0/3        unassigned      YES manual down                  down
FastEthernet0/4        unassigned      YES manual down                  down
FastEthernet0/5        unassigned      YES manual down                  down
FastEthernet0/6        unassigned      YES manual down                  down
FastEthernet0/7        unassigned      YES manual down                  down
FastEthernet0/8        unassigned      YES manual down                  down
FastEthernet0/9        unassigned      YES manual down                  down
FastEthernet0/10       unassigned      YES manual down                  down
FastEthernet0/11       unassigned      YES manual down                  down
FastEthernet0/12       unassigned      YES manual down                  down
FastEthernet0/13       unassigned      YES manual down                  down
FastEthernet0/14       unassigned      YES manual down                  down
FastEthernet0/15       unassigned      YES manual down                  down
FastEthernet0/16       unassigned      YES manual down                  down
FastEthernet0/17       unassigned      YES manual down                  down
FastEthernet0/18       unassigned      YES manual down                  down
FastEthernet0/19       unassigned      YES manual down                  down
FastEthernet0/20       unassigned      YES manual down                  down
FastEthernet0/21       unassigned      YES manual down                  down
FastEthernet0/22       unassigned      YES manual down                  down
FastEthernet0/23       unassigned      YES manual down                  down
FastEthernet0/24       unassigned      YES manual down                  down
GigabitEthernet0/1     unassigned      YES manual up                    up
GigabitEthernet0/2     unassigned      YES manual down                  down
Vlan1                  unassigned      YES manual administratively down down
```

Step 3 — Configure Hosts

Set static IP addresses per table.

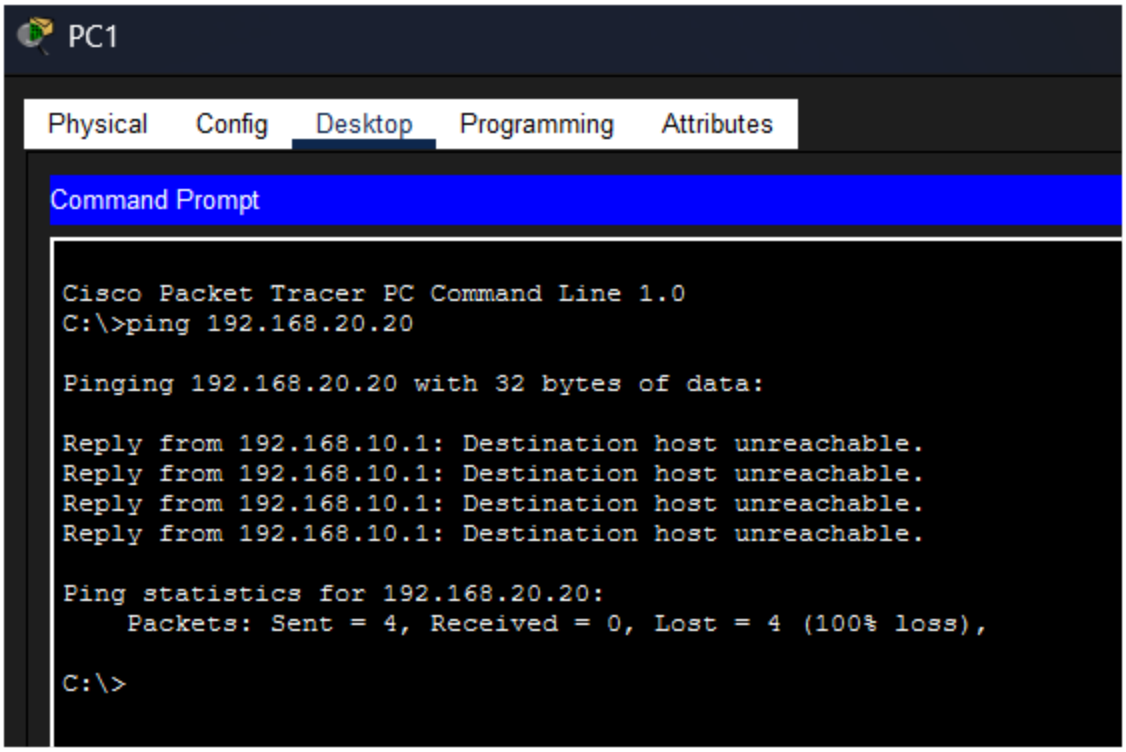




🔍 Part 2 — Verify Connectivity

From PC1:

```
None
ping 192.168.20.20
```

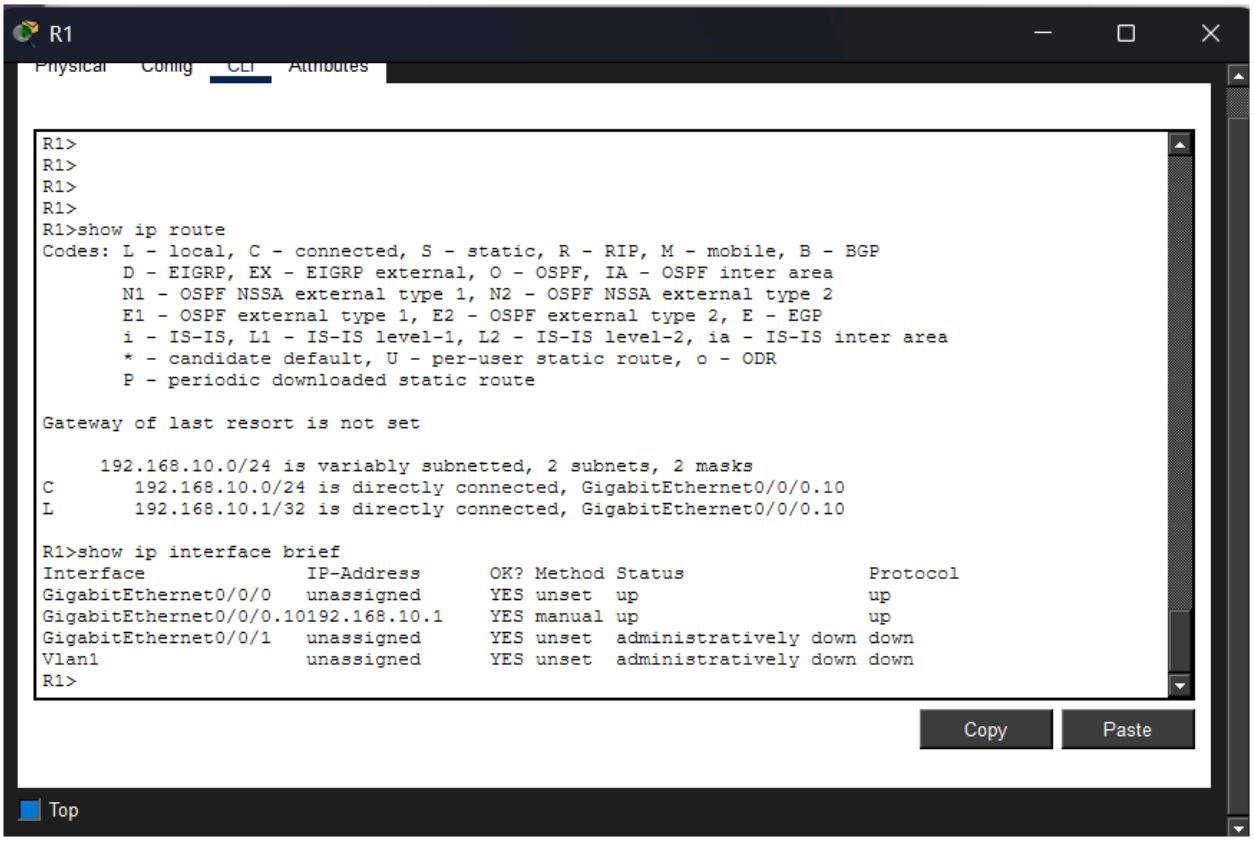


Verify:

- Successful replies
- ARP entries

Router checks:

```
None
show ip route
show ip interface brief
```



Switch checks:

None

show interfaces trunk

show mac address-table

S1

Physical Config CLI Attributes

```
S1>
S1>
S1>show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Gig0/1      on           802.1q         trunking      1

Port        Vlans allowed on trunk
Gig0/1      1-1005

Port        Vlans allowed and active in management domain
Gig0/1      1,10,20

Port        Vlans in spanning tree forwarding state and not pruned
Gig0/1      1,10,20

S1>show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    0002.160c.c801    DYNAMIC     Gig0/1
  10    0001.973c.1d10    DYNAMIC     Fa0/1
  10    0002.160c.c801    DYNAMIC     Gig0/1
S1>
```

Copy Paste

🧪 Part 3 — Deploy MIME Server

On Server (PC2)

```
None
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

```
Microsoft Windows [Version 10.0.26200.7840]
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\งานของกันตวิชญ์\network\week01-mime-typing>python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verb
ose
[11:13:37] INFO: Starting server with host=0.0.0.0, port=65432, storage=storage
[11:13:37] INFO: Server listening on 0.0.0.0:65432
[11:18:46] INFO: Connection from ('10.224.113.1', 54225)
[11:18:46] INFO: Receiving image/png (108 bytes)
[11:18:46] INFO: Saved to storage\received_7ea3.png
[11:18:46] WARNING: Connection error with ('10.224.113.1', 54225): [WinError 10053] An established connectio
n was aborted by the software in your host machine
[11:18:46] INFO: Connection with ('10.224.113.1', 54225) closed
```

On Client (PC1)

```
None
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port 65432
```

Confirm file saved.

```
C:\งานของกันตวิชญ์\network\week01-mime-typing>python client/main_enhanced.py assets/sample.png --host 10.224.11
3.1 --port 65432
[11:18:46] INFO: Starting client with 1 files to 10.224.113.1:65432
[11:18:46] INFO: Connected to 10.224.113.1:65432
[11:18:46] INFO: Sent assets/sample.png (108 bytes) as image/png
[11:18:46] INFO: Connection closed. Sent: 1, Failed: 0
[11:18:46] INFO: All files sent successfully
```

📡 Part 4 — Wireshark Capture Walkthrough on PC1 and PC1

Step 1 — Start Capture

Open Wireshark.

Select interface connected to network.

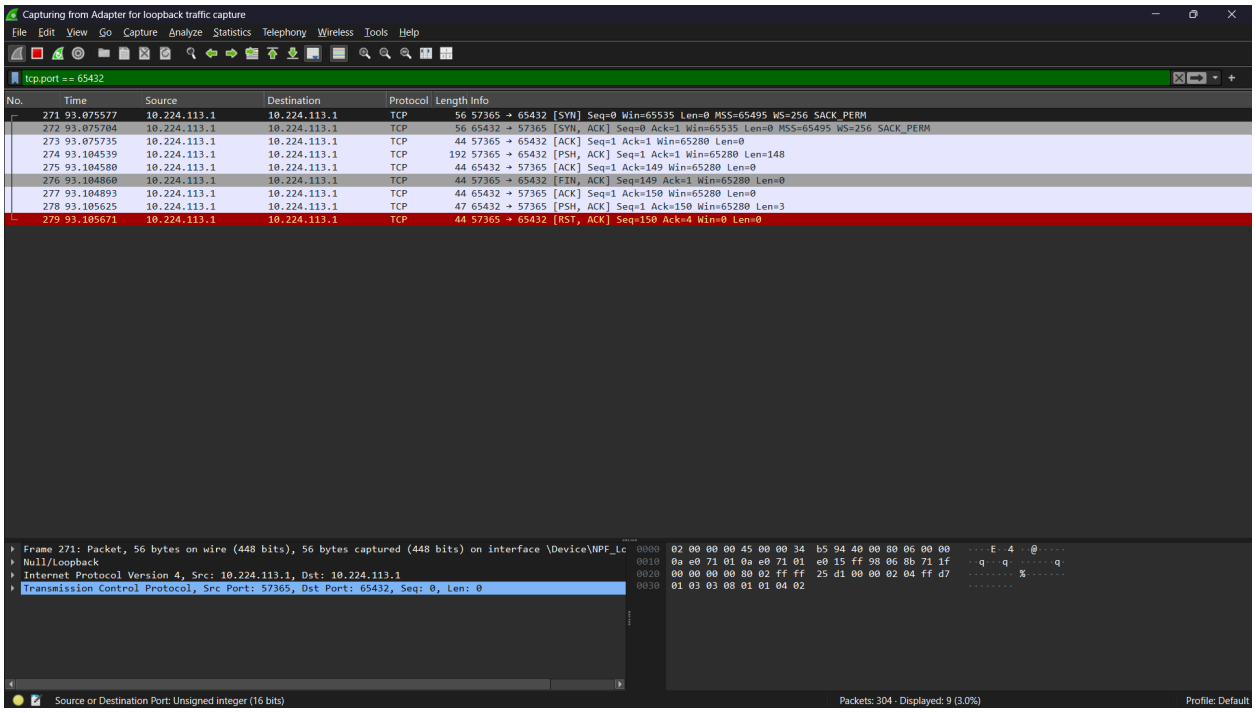
Apply capture filter:

```
None
tcp port 65432
```

Step 2 — Initiate Transfer

Send file again.

Stop capture after completion.



จากภาพ Wireshark สรุปได้ว่าการทดลองสำเร็จสมบูรณ์ครับ โดยเริ่มต้นเราเห็นคอมพิวเตอร์ทั้งสองเครื่องทักทายและสร้างการเชื่อมต่อกันได้จริง (3-Way Handshake) ใน 3 บรรทัดแรก จากนั้นก็มีการส่งข้อมูลก้อนสำคัญที่มีสถานะ PSH (Push) ซึ่งก็คือส่วนหัวไฟล์แบบ JSON ที่เราต้องการส่งตามโจทย์ และสุดท้ายเมื่อส่งเสร็จ โปรแกรมก็ทำการตัดจบการเชื่อมต่อทันที (เส้นสีแดง RST) ซึ่งทั้งหมดนี้ยืนยันว่าโค้ด Client และ Server ทำงานรับส่งข้อมูลกันได้ถูกต้องตามกลไกมาตรฐานของ TCP/IP ครับ

Step 3 — Analyze Packets

A — ARP Exchange

Look for:

- Who has 192.168.20.20?

Confirm MAC resolution.

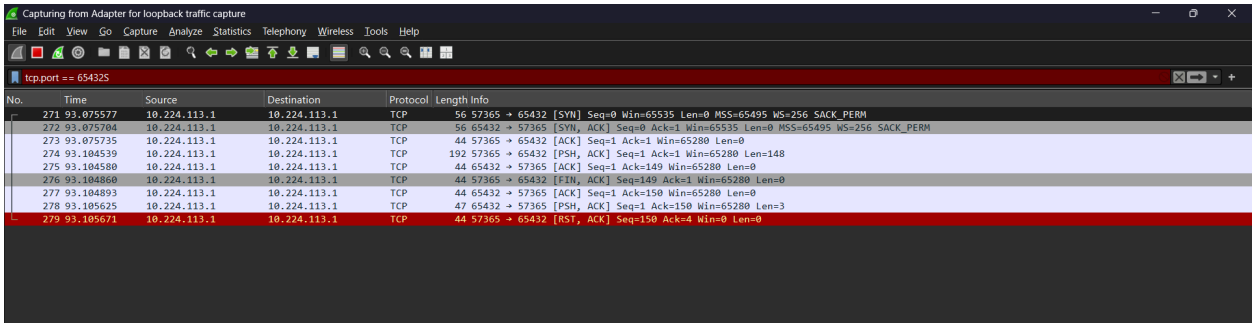
เนื่องจากการทดลองทำบนเครื่องเดียวกัน (Loopback) จึงไม่มีกระบวนการ ARP Request/Reply เกิดขึ้นจริง

B — TCP Three-Way Handshake

Identify:

1 SYN 2 SYN-ACK 3 ACK

Check sequence numbers.



จะเห็นว่า 3 บรรทัด คือกระบวนการ Three-Way Handshake

Packet 271 (SYN) เครื่อง Client ส่งคำทักทายไปบอก Server ว่าขอเชื่อมต่อหน่อย

Packet 272 (SYN, ACK) เครื่อง Server ตอบกลับมาว่า "โอเค รับทราบ (ACK) และขอเชื่อมต่อกลับด้วยนะ (SYN)

Packet 273 (ACK) เครื่อง Client ตอบกลับครั้งสุดท้ายว่า "รับทราบ การเชื่อมต่อสมบูรณ์แล้ว

C — Data Transfer

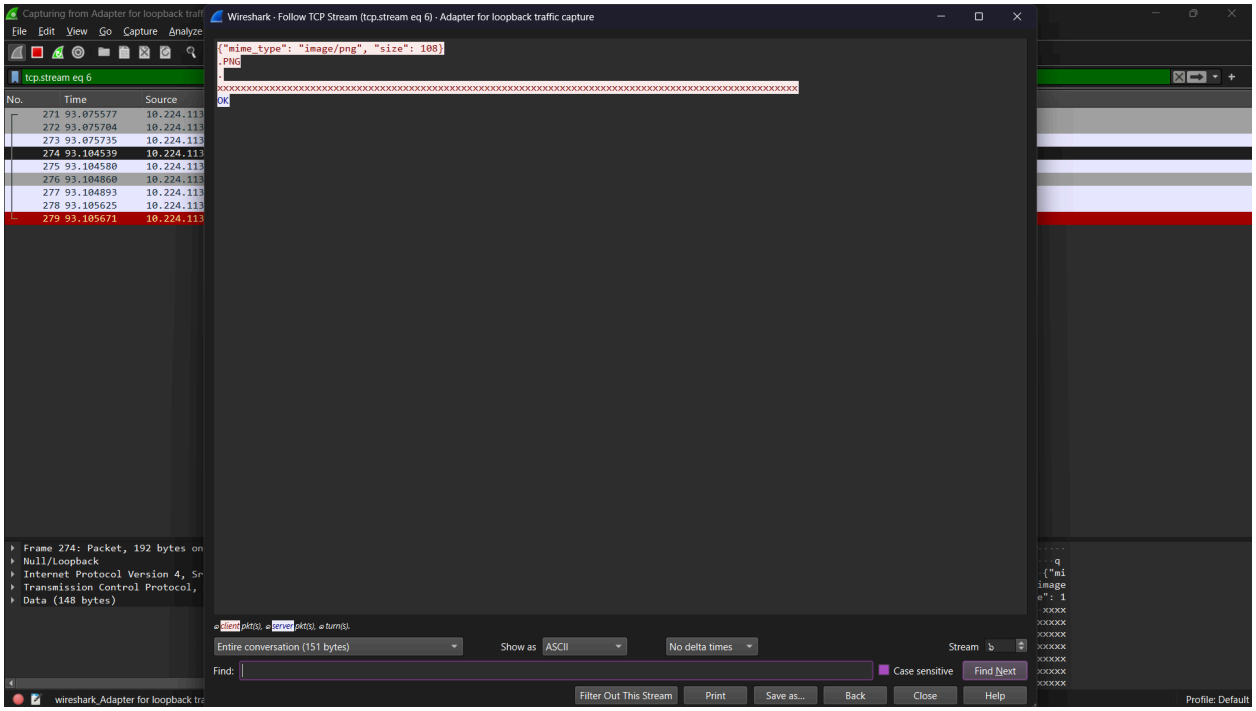
Follow stream:

Right click ⟶ Follow ⟶ TCP Stream.

You should see JSON header similar to:

```
None
{"mime_type":"image/png","size":12345}
```

Followed by binary.



D — VLAN Tags

If capturing on trunk:

Look for:

```
None
802.1Q Virtual LAN
```

Note VLAN ID.

ในส่วนของการวิเคราะห์ VLAN Tags จะไม่พบข้อมูล 802.1Q Header ใน Wireshark เนื่องจากเป็นการทดสอบแบบจำลองบนเครื่องเดียว (Localhost) ข้อมูลจึงไม่ได้ถูกส่งผ่าน Trunk Link ของ Switch จริง ทำให้ไม่มีการ Encapsulation แบบ 802.1Q เกิดขึ้น แต่ถ้าทดลองจริง จะเห็นฟิลด์ 802.1Q Virtual LAN แทรกอยู่ระหว่าง Ethernet และ IP Heade ภายในจะระบุ VLAN ID เป็น 10 หรือ 20 ตามที่เราตั้งค่าไว้

🔬 Packet Dissection Guide

Layer	Field	What to Observe
Ethernet	Src/Dst MAC	Frame direction
802.1Q	VLAN ID	Segmentation
IP	TTL	Routing behavior
TCP	Flags	Reliability
Payload	JSON	MIME metadata

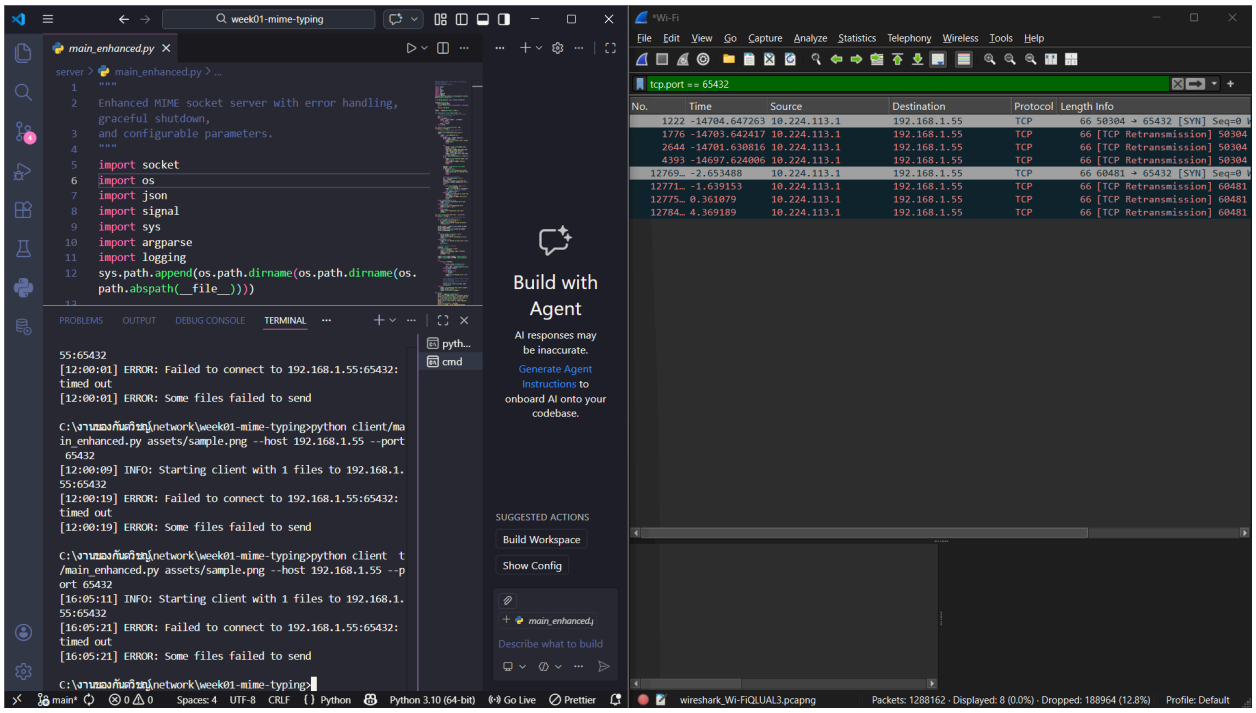
🛠️ Part 5 — Troubleshooting Labs

Scenario 1 — Broken Routing

Shutdown subinterface.

Observe:

- Ping failure
- TCP timeout



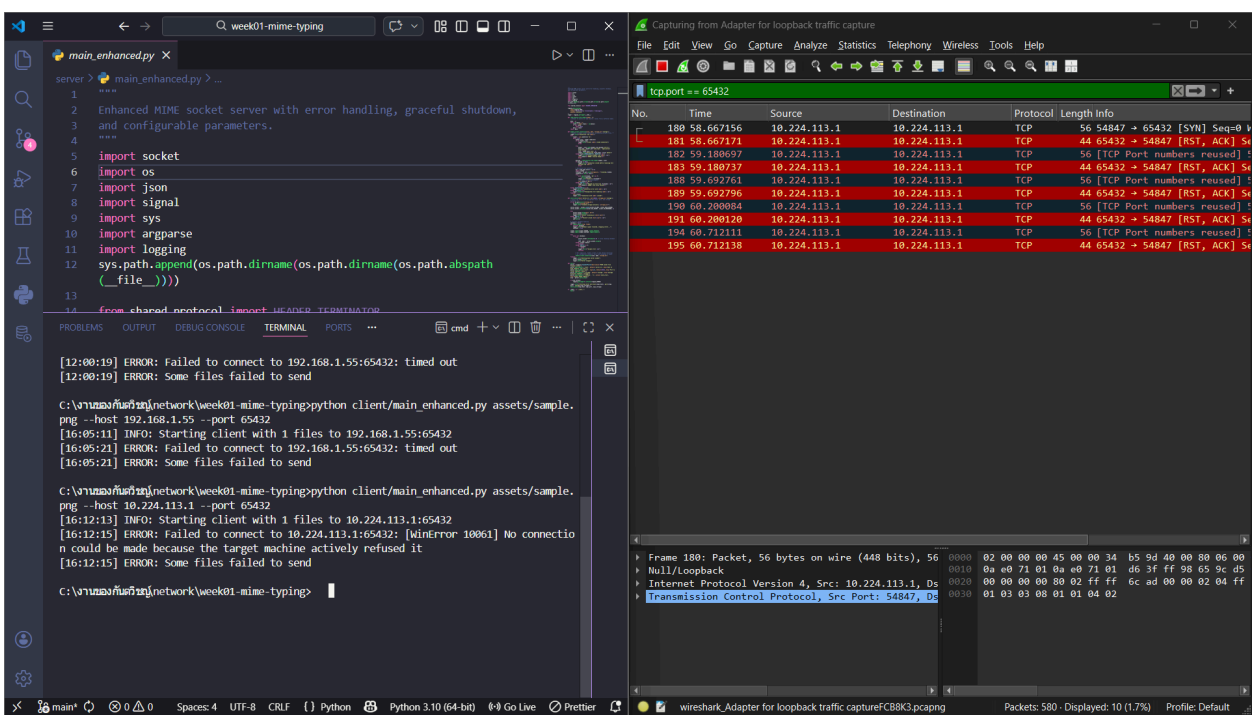
กำหนดให้ Client พยายามเชื่อมต่อไปยัง IP Address ที่ไม่มีอยู่จริงในเครือข่าย (192.168.1.55) จะเห็นว่าฝั่ง Client มีการส่งแพ็กเก็ต TCP [SYN] เพื่อขอสร้างการเชื่อมต่อ แต่เนื่องจากไม่สามารถจัดส่งแพ็กเก็ตไปยังปลายทางได้ จึงไม่มีการตอบรับกลับมา ส่งผลให้เกิด TCP Timeout และระบบกลไกของ TCP ได้ทำการส่งข้อมูลซ้ำ แสดงผลเป็นแพ็กเก็ตสถานะ [TCP Retransmission] ซึ่งสอดคล้องกับพฤติกรรมเมื่อเกิดปัญหา Routing

Scenario 2 — Firewall Block

Block port.

Observe:

- SYN retransmissions

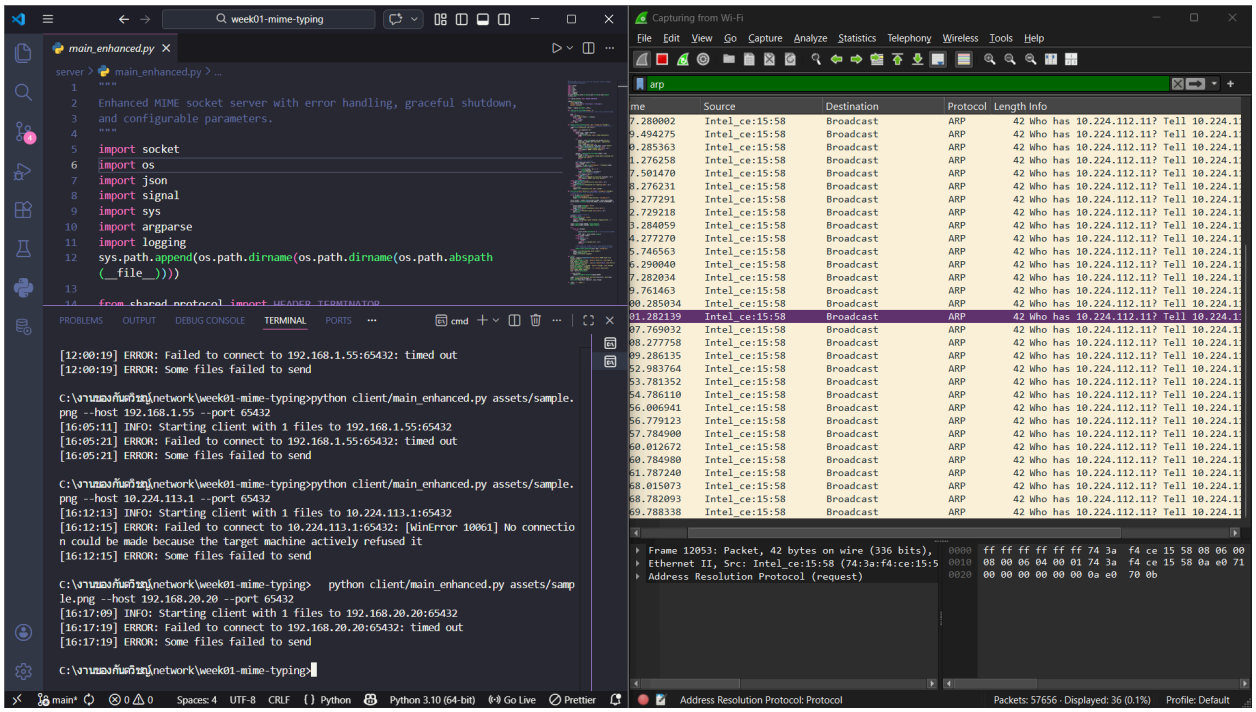


จากภาพ Wireshark เมื่อ Client พยายามส่งแพ็กเก็ต SYN เพื่อเริ่ม Three-Way Handshake ระบบปฏิบัติการปลายทางพบว่าไม่มี Service รอรับอยู่ที่พอร์ตดังกล่าว จึงตอบกลับด้วยแพ็กเก็ตสถานะ [RST, ACK] (Reset) เพื่อปฏิเสธการเชื่อมต่อทันที

Scenario 3 — Wrong Gateway

Observe:

- ARP incomplete



จากภาพ Wireshark พบว่าโปรโตคอล ARP พยายามทำงานโดยส่งแพ็กเก็ตแบบ Broadcast เพื่อถามหา MAC Address (Who has...?) แต่ไม่มีอุปกรณ์ใดตอบกลับ ทำให้กระบวนการแปลงหมายเลข IP เป็น MAC Address ไม่สมบูรณ์ เกิด

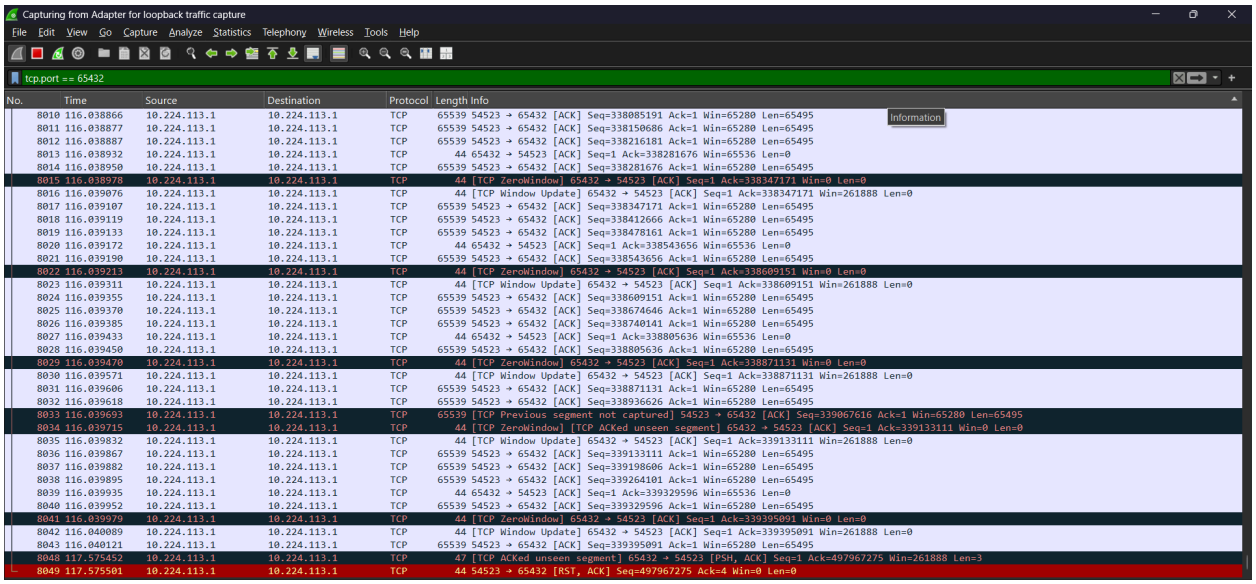
สถานะ ARP Incomplete เมื่อไม่สามารถทำงานใน Layer 2 ได้สำเร็จ กระบวนการของ TCP ในระดับที่สูงกว่าจึงไม่สามารถเริ่มต้นได้

Scenario 4 — Packet Loss Simulation

Drop packets.

Observe:

- TCP retransmissions
- Window adjustment



จาก Wireshark จะเห็นว่าเมื่อเกิดการสูญหายของแพ็กเก็ตหรือเครือข่ายมีปัญหา (สังเกตจาก [TCP Previous segment not captured]) โปรโตคอล TCP จะใช้กลไกการควบคุมการไหลของข้อมูล (Flow Control) โดยฝั่งรับจะส่งแพ็กเก็ตสถานะ [TCP ZeroWindow] เพื่อแจ้งให้ฝั่งส่งหยุดส่งข้อมูลชั่วคราว และตามด้วย [TCP Window Update] เมื่อพร้อมรับข้อมูลต่อ พฤติกรรมนี้

คือการทำ Window adjustment เพื่อป้องกันไม่ให้เครือข่ายแออัดจนล่ม ซึ่งเป็นการแสดงให้เห็นถึงความน่าเชื่อถือ (Reliability) ของโปรโตคอล TCP ในการจัดการกับสภาวะเครือข่ายที่ไม่เสถียร

📝 Lab Report Requirements

Include:

- Screenshots of configurations
- Ping results
- Wireshark capture screenshots
- JSON header evidence
- Encapsulation explanation
- Troubleshooting notes

🎓 Assessment Questions

1. Why does TCP retransmit lost segments?
    - เพื่อให้มั่นใจว่าข้อมูลถูกส่งถึงปลายทางอย่างครบถ้วนและถูกต้อง (Reliability) โดย TCP จะใช้กลไกการตอบรับ (Acknowledgement) หากผู้ส่งไม่ได้รับเลข ACK กลับมาภายในเวลาที่กำหนด จะถือว่าข้อมูลสูญหายและทำการส่งใหม่
2. How does VLAN tagging affect broadcast domains?
    - VLAN tagging (802.1Q) ช่วยในการแบ่ง Broadcast Domain ขนาดใหญ่ออกเป็นส่วนย่อยๆ ตาม ID ของ VLAN นั้นๆ ทำให้ Traffic ประเภท Broadcast ของ VLAN หนึ่ง ไม่ไปรบกวนหรือแพร่กระจายไปยัง VLAN อื่นที่ไม่ได้อยู่ในกลุ่มเดียวกัน
3. What role does MIME play compared to TCP headers?
    - TCP Header: ทำหน้าที่ควบคุมการส่งข้อมูลในระดับ Transport Layer เช่น การระบุ Port และลำดับของข้อมูล

- MIME: ทำหน้าที่ระบุประเภทและรูปแบบของ "เนื้อหา" (Content-Type) ภายใน Payload เช่น การระบุว่าเป็นไฟล์รูปภาพ ข้อความ หรือวิดีโอ เพื่อให้แอปพลิเคชันปลายทางประมวลผลข้อมูลได้ถูกต้อง

4. Where does routing decision occur?
    - การตัดสินใจเลือกเส้นทาง (Routing Decision) เกิดขึ้นที่ Router (หรือ Layer 3 Device) โดย Router จะตรวจสอบ IP Address ปลายทางเทียบกับ Routing Table เพื่อหาพอร์ตที่เหมาะสมในการส่งข้อมูลต่อไป
5. Why does ARP precede TCP communication?
    - เพราะในระบบเครือข่ายท้องถิ่น (LAN) การส่งข้อมูลจริงต้องระบุ MAC Address ของอุปกรณ์ปลายทาง แต่เรามักจะรู้เพียง IP Address ดังนั้น ARP จึงต้องทำงานก่อนเพื่อค้นหา (Resolve) ว่า IP ปลายทางนั้นมี MAC Address เป็นอะไร จึงจะสามารถสร้าง Frame ข้อมูลเพื่อเริ่มขั้นตอน TCP Three-way Handshake ได้

📊 Reflection Exercise

Explain:

How does application meaning survive multiple encapsulation layers?

ความหมายของข้อมูลในระดับแอปพลิเคชัน (Application Meaning) สามารถอยู่รอดผ่านการห่อหุ้มหลายชั้น (Encapsulation) ได้ด้วยกลไก **"Layer Independence"** และ **"Headers Control"** ดังนี้ครับ:

- **การแยกส่วนหน้าที่ (Layer Independence):** แต่ละชั้นใน OSI Model จะไม่ยุ่งกับ "เนื้อหา" (Payload) ของชั้นที่อยู่เหนือกว่า โดยจะมองข้อมูลจากชั้นบนเป็นเพียงก้อนข้อมูล (Data) ก้อนหนึ่ง แล้วนำมาใส่ "ซอง" (Header) ของตัวเองทับลงไปเท่านั้น
- **บทบาทของ Header:** ข้อมูลที่ใช้สื่อสารในแต่ละระดับจะถูกเก็บไว้ใน Header ของชั้นนั้นๆ เช่น TCP Header จะเก็บเลข Port เพื่อบอกว่าข้อมูลนี้เป็นของแอปไหน, IP Header จะเก็บเลข IP เพื่อบอกว่าส่งไปที่เครื่องใด ซึ่งข้อมูลเหล่านี้จะไม่ไปปนกับเนื้อหาจริงของแอปพลิเคชัน
- **กระบวนการ Decapsulation:** เมื่อข้อมูลถึงปลายทาง อุปกรณ์ฝั่งรับจะทำกระบวนการย้อนกลับ (Decapsulation) โดยค่อยๆ แกะ "ซอง" ออกทีละชั้นตามลำดับ จากล่างขึ้นบน จนเหลือเพียงก้อนข้อมูลเดิมที่ถูกส่งมาจากชั้น Application ของต้นทาง
- **มาตรฐานโปรโตคอล (Standardization):** การที่ทั้งสองฝั่งใช้โปรโตคอลชุดเดียวกัน (เช่น HTTP, TCP, IP) ทำให้มั่นใจได้ว่า "การตีความ" ข้อมูลในแต่ละซองจะตรงกันเสมอ ความหมายของข้อมูลจึงไม่ผิดเพี้ยนไป

⚠️ Common Pitfalls

- Server bound to localhost only
- Missing trunk
- Wrong VLAN
- OS firewall enabled
- Capture on wrong interface

🚀 Extension Activities

- Add TLS encryption
- Measure throughput vs latency
- Capture checksum errors
- Add checksum field to protocol
- Containerize server
- Compare with HTTP transfer